WAC
Windows Artefacts Collector

CENTRE D'EXCELLENCE CYBERDEFENSE AEROSPATIALE

Créé par le Centre d'Excellence Cyberdéfense Aérospatiale (CEC)

Dossier de validation
WAC (Windows Artefacts Collector)

Logiciel de prélèvement d'artefacts Windows pour l'investigation numérique



ARTEFACTS SYSTÈME

FICHIERS UTILISATEUR

PREUVES NUMÉRIQUES

UN OUTIL AU SERVICE DE L'INVESTIGATION NUMÉRIQUE

WAC — Windows Artefacts Collector

Dossier de validation

Version du logiciel	2.0.1
Édition du document	septembre 2026
Éditeur	Centre d'Excellence Cyberdéfense Aérospatiale (CEC)
	École de l'air et de l'espace, base aérienne 701, Salon-de-Provence

Ce document recense les contrôles qui établissent la qualité de WAC : ceux que WAC exécute lui-même à chaque collecte, les programmes de test qui confrontent ses décodeurs à une référence, et les contrôles automatiques de ses sorties. Pour chacun, il dit ce qui est comparé, à quoi, le résultat de référence et les défauts qu'il a déjà pris. Il dit aussi ce que ces contrôles ne garantissent pas. La façon de lancer chaque test est dans le *Guide des tests*.

Table des matières

1 La démarche **2**

1.1 Pourquoi un JSON valide ne prouve rien 2

1.2 Quatre niveaux de contrôle 2

1.3 Les règles de développement qui s'y rattachent 2

2 Les contrôles intégrés à WAC **4**

2.1 Avant toute extraction 4

2.2 Lecture brute du volume NTFS 4

2.3 Ruches du registre 5

2.4 Intégrité des pièces : la consigne 5

2.5 Authenticité des binaires 6

2.6 Journaux d'événements 6

2.7 Décodage des artefacts 7

3 Les programmes de test **8**

3.1 Harnais de format 8

3.2 Conversions : le détail de `system_conversions_test` 11

3.3 Harnais de robustesse 11

4 Le cycle automatique et ses contrôles **13**

4.1 Le cycle 13

4.2 Les contrôles de `check-json.py` 13

4.3 Résultat de référence 23

5 Les recoupements sur une collecte réelle **25**

6 Ce que ces contrôles garantissent **26**

6.1 Garanties 26

6.2 Limites 26

CHAPITRE 1

La démarche

1.1 Pourquoi un JSON valide ne prouve rien

WAC produit des fichiers JSON destinés à une expertise. Qu'ils soient valides ne dit presque rien de leur justesse : un champ lu au mauvais décalage, une date interprétée dans le mauvais fuseau, un nom décodé dans la mauvaise page de code, une décompression fausse donnent tous un JSON parfaitement valide, et faux. Les défauts les plus graves relevés dans WAC étaient tous de cette sorte. Aucun ne se voyait à l'exécution ; chacun n'est apparu qu'en confrontant une valeur à une **source indépendante**.

La validation de WAC repose donc sur ce principe : toute donnée qui peut être recoupée l'est, par un moyen qui ne partage rien avec le code vérifié.

1.2 Quatre niveaux de contrôle

Les contrôles intégrés à WAC (chapitre 2) : exécutés à chaque collecte, sur la machine examinée. Ils refusent une donnée incohérente plutôt que de la publier, et attestent l'intégrité des pièces prélevées.

Les programmes de test (chapitre 3) : seize exécutables qui confrontent chaque décodeur à une référence (la fonction de Microsoft qu'il remplace, une norme publiée, une autre implémentation), ou qui prouvent qu'un analyseur ne lit jamais hors de son tampon.

Le cycle automatique et ses contrôles (chapitre 4) : une collecte complète dans une machine virtuelle Windows 11, puis `check-json.py`, qui vérifie la validité des sorties et les confronte entre elles et à des sources externes.

Les recoupements sur une collecte réelle (chapitre 5) : ce qu'une machine virtuelle neuve ne contient pas (journaux anciens, fichiers fragmentés, noms exotiques) est vérifié sur les données d'une vraie machine, traitées en local.

1.3 Les règles de développement qui s'y rattachent

1. **Un correctif s'accompagne du test qui aurait pris le défaut**, et ce test est vérifié dans les deux sens : il passe sur le code corrigé, et il échoue quand on réintroduit le défaut. Un test qui n'a jamais été vu échouer ne prouve rien.
2. **La compilation se fait sans avertissement**, options `-Wall -Wextra` activées. Les avertissements ont d'ailleurs servi de liste de travaux : chacun signalait un champ

décodé puis perdu.

3. **Après toute modification, la nouvelle collecte est comparée à la précédente**, artefact par artefact : un artefact passé de 1 522 entrées à 573 n'allume aucun voyant, seule la comparaison le montre.
4. **Toute entrée est hostile**, y compris un fichier du disque examiné : chaque lecture est bornée par la taille réelle du tampon, jamais par une longueur annoncée dans la donnée.
5. **Un champ sans valeur n'est pas émis** : une clé vide ou nulle se lirait comme une donnée relevée, ou comme une lecture ratée.

CHAPITRE 2

Les contrôles intégrés à WAC

Ces contrôles font partie du programme de collecte. Ils s'exécutent à chaque collecte et leur résultat est consigné dans `investigation.json`, opération par opération, avec son code de retour et la trace qu'elle laisse sur le système examiné.

2.1 Avant toute extraction

— version de Windows (10 ou ultérieure) et droits d'administrateur effectifs (jeton élevé) ;

— emplacement de collecte utilisable et place disponible, vérifiés *avant* la première extraction : une collecte interrompue faute de place laisserait une consigne incomplète.

2.2 Lecture brute du volume NTFS

Les fichiers verrouillés (ruches, journaux) sont lus directement sur le volume, sans passer par le système de fichiers de Windows. Chaque structure lue est vérifiée avant d'être utilisée :

— signature `FILE` de chaque enregistrement de la `$MFT` et `INDX` de chaque bloc d'index, puis vérification des *fixups* (tableau de séquence de mise à jour) : chaque tranche de 512 octets doit se terminer par le numéro de séquence de l'enregistrement. Un enregistrement *déchiré* par une écriture interrompue, mélange de deux versions, est refusé au lieu d'être lu comme une donnée plausible ;

— décompression LZNT1 et WOF bornée par la taille attendue de chaque unité : une unité qui ne produit pas le nombre d'octets prévu est une erreur, pas une donnée tronquée en silence ;

— taille extraite comparée à la taille déclarée par l'attribut `$DATA` : leur écart, publié dans le manifeste, révèle une extraction tronquée qu'une empreinte seule ne montrerait pas ;

— attributs cherchés aussi dans les enregistrements d'extension (`$ATTRIBUTE_LIST`) : un répertoire dont NTFS a déplacé le `$INDEX_ROOT`, cas de milliers de composants WinSxS, était déclaré illisible (4 569 répertoires d'un Windows 11) ;

— blocs d'index libérés ignorés, d'après le `$BITMAP` de l'index : ils peuvent garder des entrées périmées ;

— chaque lecture d'attribut bornée par la taille réelle de l'attribut, `$STANDARD_INFORMATION` compris ;

— dates du fichier source relevées dans `$STANDARD_INFORMATION` *et* dans `$FILE_NAME` : un

programme qui falsifie les premières (*timestomping*) ne touche pas les secondes, et l'écart se voit au manifeste ;

— volume ouvert sans cache (`FILE_FLAG_NO_BUFFERING`) : chaque lecture atteint le disque, jamais une copie en mémoire d'un secteur lu plus tôt ; même durée de collecte mesurée (194 s contre 198 s).

2.3 Ruches du registre

WAC lit les ruches avec son propre lecteur, lié dans l'exécutable, et non avec une bibliothèque de la machine examinée.

— bloc de base : signature `regf`, somme de contrôle (OU exclusif des 127 premiers mots), numéros de séquence primaire et secondaire. Une ruche dont les journaux n'ont pas été rejoués est **refusée** : lue telle quelle, elle donnerait des clés périmées sans aucune erreur ;

— rejeu des journaux de transactions (`.LOG1`/`.LOG2`) : signature et chaînage des séquences de chaque entrée ; une entrée hors chaîne est écartée et comptée ; un journal d'annulation est écrit avant toute modification, pour pouvoir reconstituer la copie brute. Le numéro de séquence d'une ruche est celui de la prochaine entrée qu'elle attend : l'entrée de même numéro est donc appliquée, et l'en-tête prend ensuite le numéro suivant la dernière entrée et la taille que celle-ci annonce. Le résultat est confronté à Windows : la même ruche et ses journaux, chargés par `reg load`, donnent les mêmes clés, les mêmes valeurs et la même empreinte de l'ensemble que la ruche rejouée par WAC ;

— chaque décalage lu dans la ruche est vérifié contre la taille réelle du fichier, et une sous-clé qui pointerait vers l'une de ses ancêtres est refusée : un parcours ne peut pas boucler sur une ruche forgée.

2.4 Intégrité des pièces : la consigne

Toute pièce extraite est déposée dans une *consigne* qui n'est plus jamais rouverte en écriture ; tout le travail se fait sur une seconde copie.

— trois empreintes par pièce (MD5, SHA-1, SHA-256), calculées pendant la lecture du volume ;

— la copie de travail est vérifiée par son SHA-256 contre ce qui a été lu sur le volume, et non contre une relecture de la consigne ;

— le manifeste qui identifie les pièces est scellé par un second fichier portant son empreinte : un manifeste retouché se détecte ;

— un échec d'extraction est consigné lui aussi : une pièce absente du manifeste se lirait comme jamais cherchée ;

— les dates d'un artefact fichier (Prefetch, raccourci, liste de sauts) sont celles du fichier source, relevées au manifeste, jamais celles de la copie de travail, qui sont celles de la collecte ;

— une conversion séparée (`--convert`) vérifie le sceau et le SHA-256 de chaque pièce *avant* toute lecture, refuse la consigne au premier écart, et n'accepte que des chemins de pièces qui restent dans la consigne.

Un tiers peut vérifier la consigne sans WAC, avec les scripts `verify-exhibits.ps1` (messages en anglais) et `verifier-consigne.ps1` (messages en français), livrés avec la documentation utilisateur, et leurs équivalents Linux `verify-exhibits.sh` et `verifier-consigne.sh`, qui calculent les empreintes par `sha256sum` et non par WAC. Tous contrôlent le sceau puis l'empreinte de chaque pièce ; un binaire authentifié lu mais non copié (« empreinte seule ») est compté à part, faute de fichier. Le manifeste venant de la machine examinée, un chemin de pièce qui sort de la collecte (`..`, chemin absolu, lien symbolique pour les `.sh`) est refusé sans être lu : les premières versions des `.ps1` comptaient comme vérifié un fichier du poste d'analyse désigné par `exhibits\..\..\..`, pourvu que son empreinte figure au manifeste — défaut pris par une contre-épreuve dans la machine de test. `verify_exhibits_test.sh` vérifie les deux `.sh` sur des collectes forgées (23 contrôles).

2.5 Authenticité des binaires

Avec l'option `--binary`, WAC prélève les binaires cités par les artefacts, sauf ceux dont il établit qu'ils sont signés par Microsoft. Cette décision repose sur une vérification complète, faite par WAC lui-même : signature RSA de chaque catalogue de sécurité, chaîne de certificats jusqu'à une racine Microsoft, empreinte Authenticode du fichier retrouvée dans un catalogue vérifié, ou signature intégrée. Les catalogues lus sont ceux de `CatRoot`, des composants (`WinSxS\Catalogs`) et des paquets de maintenance ; les signataires acceptés, ceux de Microsoft pour son propre code, `.NET` compris. Les fichiers d'une application du Store sont vérifiés par la signature de leur paquet et la carte de ses blocs, bloc de 64 Kio par bloc. Un fichier dont l'authenticité n'est pas établie est prélevé : le doute conduit à collecter, jamais à écarter. Les catalogues, signatures et cartes qui ont justifié un non-prélèvement sont consignés ; chaque PE porte au manifeste sa version (`TimeDateStamp`, `SizeOfImage`), clé du serveur de symboles de Microsoft qui permet d'en retrouver le contenu identique. Avec `--binary-all`, tout est prélevé, et le verdict est consigné pour chaque exécutable.

2.6 Journaux d'événements

Signature du fichier, de chaque bloc (*chunk*) et de chaque enregistrement. Tous les blocs physiques du fichier sont parcourus, et pas seulement ceux que l'en-tête déclare : un journal mal fermé livre ainsi les enregistrements qu'il ne compte pas. Un bloc illisible est signalé et sauté ; il n'interrompt plus la lecture du fichier.

Messages : les écarts résiduels avec Windows

Le cycle compare le message de chaque sorte d'événement à celui que Windows rend lui-même (`check-json.py`, seuil de 90 % de sortes identiques ; mesuré : 887 sur 898). Les dates insérées sont comparées sans les marques de direction (U+200E) que Windows y place et à la centaine de nanosecondes. Le même score est obtenu quand la collecte (`--collect`) et la conversion sont séparées. Les écarts qui restent sont expliqués, sorte par sorte :

— **une donnée hexadécimale reliée à une table** : Windows n'applique pas la table selon le manifeste. `DehPhase = 0x1` est « PerMachine » selon le manifeste d'UndockedDeh, Windows écrit « PerUser » (libellé de 0) ; une table de drapeaux sur `0x2` reste « 0x2 » chez Windows (SMB 30950, 1080). WAC suit le manifeste (4 sortes) ;

— **satellite plus ancien que son binaire** : le `fr-FR\audioses.dll.mui` de la machine (version 10.0.26100.1) porte ces trois messages en anglais, et le texte défectueux « Signature utilisée :chiffrement » est celui du `mrxsmb.sys.mui` présent sur le disque ; Windows affiche un texte français absent de ces fichiers, dont l'origine n'a pas été trouvée (4 sortes) ;

— **date FILETIME nulle** : Windows écrit 1601-01-01, WAC n'insère rien (Wininit 25) ;

— **sans message chez WAC** : un événement dont Windows n'a pu mettre les données en forme (`ProcessingErrorData`, AppModel-Runtime 219) et Perflib 0 (2 sortes).

2.7 Décodage des artefacts

— toute lecture est bornée par la taille du tampon ; un nombre d'éléments déraisonnable dans une liste de valeurs (au-delà de 65 536) est refusé plutôt que parcouru ;

— une date impossible (mois 13, heure 25) ou hors de la plage d'un `FILETIME` donne une date absente, donc non émise, et jamais une valeur quelconque ;

— chaque date locale porte le décalage en vigueur *à sa date*, selon les règles d'heure d'été de la machine examinée, année par année ;

— les SID sont nommés à partir des preuves seules (SAM, SID bien connus, SID de services, profils), jamais en interrogeant le système ou le réseau : `LookupAccountSidW` interrogeait le contrôleur de domaine pour un SID inconnu de la machine, une trace de la collecte sur une autre machine. La compilation échoue si `WAC.exe` importe une fonction de résolution de noms ;

— un GUID inconnu de la table de référence est nommé d'après les ruches de la machine examinée (classes de la machine, dossiers connus, classes de chaque utilisateur) ; inconnu partout, il est publié sans nom, jamais avec un texte de remplacement.

CHAPITRE 3

Les programmes de test

Seize exécutables, exclus de la compilation de WAC par leur suffixe `_test.cpp` et construits par `build-windows.sh --test`. Chacun vise un composant et le confronte à un juge qui ne partage rien avec lui.

3.1 Harnais de format

Programme	Juge	Résultat de référence
`sha_test`	vecteurs de FIPS 180-4, implémentation indépendante	SHA-1 et SHA-256 conformes, y compris sur les longueurs 55 à 128 octets qui exercent le remplissage
`lznt1_test`	compresseur de Microsoft (`RtlCompressBuffer`)	16 unités compressées sur 16 conformes, 1 048 576 octets identiques
`xpress_test`	compresseur de Microsoft	137 blocs compressés sur 137 conformes, octet pour octet
`lzx_test`	compresseur de Microsoft (`compact /exe:lzx`)	646 blocs sur 646 conformes (trois fichiers) ; 235 707 entrées tronquées ou altérées sans lecture hors borne (sanitizers)
`wevt_test`	fichiers d'un vrai fournisseur d'événements	chaîne complète de production des messages vérifiée, marques de substitution comprises
`event_messages_test`	ruche SOFTWARE et binaires de la machine	même chaîne, étape par étape, à partir de la ruche
`evtx_test`	`python-evtx`, implémentation indépendante	enregistrement par enregistrement ; chaîne complète identique en flux et en mémoire (pic de 26 Mo contre 1 281 Mo)
`offline_registry_test`	`offreg.dll` de Microsoft	18 ruches, dont dix d'une machine réelle : 1 273 652 clés et 2 456 139 valeurs identiques
`system_conversions_test`	fonctions Windows remplacées	9 861 570 comparaisons, aucun écart (détail ci-dessous)

Programme	Juge	Résultat de référence
`consigne_test`	la consigne elle-même, avant et après	consigne identique octet pour octet après modification de la copie de travail ; sceau conforme ; chemins de pièces confinés à la consigne
`ntfs_fixup_test`	enregistrements construits, vérification indépendante	enregistrements sains restitués à l'octet, enregistrements déchirés et tableaux malformés refusés, 20 000 enregistrements aléatoires contre une page interdite : 44 610 contrôles, aucun échec
`mft_test`	`ntfsinfo` (ntfs-3g), implémentation NTFS indépendante	volume formaté par `mkntfs` : nom, parent, taille, huit dates et flux nommés identiques pour chaque enregistrement ; 278 contrôles, second volume compris, aucun échec
`usn_journal_test`	structures du compilateur (`USN_RECORD` de MinGW, version 3 d'après Microsoft)	champs restitués tels qu'écrits, numéros égaux aux positions dans le flux entier, chemins par la `$MFT`, enregistrements abîmés contre une page interdite ; 27 contrôles, aucun échec. A révélé que la séquence de la racine n'était pas vérifiée : un parent désigné par une séquence périmée donnait un chemin affirmé à tort
`fat_test` et `vmtest/lib/fat-compare.py`	dissect.fat (Fox-IT) ; la liste que Windows donne des clés	images FAT12, FAT16 (mkfs.vfat, pyfatfs), FAT32 et exFAT (formatées par Windows) : entrées présentes identiques à celles de dissect.fat, noms longs accentués, entrées supprimées retrouvées ; la copie des métadonnées rend les mêmes entrées que le volume ; chaîne FAT en boucle terminée ; 17 contrôles, aucun échec. A révélé les noms longs supprimés tronqués par une entrée plus récente, désormais signalés

Programme	Juge	Résultat de référence
`ese_test` et `vmtest/lib/ese-compare.py`	dissect.esedb (Fox-IT), lecteur ESE écrit indépendamment ; `Get-BitsTransfer`	enregistrements construits selon le format (colonnes fixes, variables, étiquetées, multivaluées), longueurs mensongères refusées, blobs BITS ; sur `SRUDB.dat` et `qmgr.db` de la VM (pages de 4 et 16 Kio, valeurs longues à identifiant de 64 bits), 1 064 268 valeurs identiques à celles de dissect.esedb ; 19 contrôles, aucun échec. A révélé les identifiants de valeurs longues de 8 octets
`cim_test`	condensat d'un nom relevé dans l'`INDEX.BTR` de la VM ; `Get-CimInstance`	classe, ancêtre et instance construits selon le format (défaut hérité, chaînes d'un octet et UTF-16, propriété non renseignée), longueurs mensongères refusées ; sur le dépôt de la VM, chaque abonnement identique à ce que rend WMI ; 14 contrôles, 15 avec ce dépôt, aucun échec
`file_artefacts_test`	description publiée de chaque format	historique PowerShell, `setupapi.dev.log`, `$I` de la corbeille (versions 1 et 2), `hosts`, `Report.wer`, MPLog de Defender : cas ordinaires et hostiles (`$I` tronqué, longueur mensongère, date impossible), flux `Zone.Identifier`, DetectionHistory et entrées de quarantaine (clé RC4 éprouvée sur un en-tête écrit par Defender, longueurs mensongères refusées) ; 62 contrôles, aucun échec
`shell_items_test`	octets écrits par l'Explorateur de la VM ; libfwsi et la bibliothèque Lnk de SBECmd	éléments de shellbag d'un contenu d'archive : ZIP (dossier réel, fichier avec tailles, date absente, disposition XP) et 7z / tar.gz lus par libarchive (éléments réels, date UTC), toutes leurs troncatures contre une page interdite ; 683 contrôles, aucun échec
`json_test`	l'écrivain JSON de WAC	aller-retour identique, 26 fichiers d'une collecte relus et réécrits à l'octet, textes malformés refusés, 200 000 mutations sans lecture hors borne (sanitizers)

Programme	Juge	Résultat de référence
`raw_hive_test`	`reg load` de Windows	la ruche extraite est refusée avant réparation et acceptée après : le test échoue dans les deux sens s'il le faut
`authenticode_test`	`Get-AuthenticodeSignature`	2 233 binaires : 2 126 authentifiés par les deux, 99 prélevés par les deux, aucun fichier accepté par WAC et refusé par Windows ; 462 scripts PowerShell et 11 scripts WSH conformes

3.2 Conversions : le détail de `system_conversions_test`

WAC ne dépend que de cinq bibliothèques système (`ADVAPI32`, `KERNEL32`, `msvcrt`, `Secur32`, `WTSAPI32`), nécessaires pour interroger le système en marche. Les conversions qu'il demandait autrefois à d'autres bibliothèques de la machine examinée sont faites par son propre code, et ce programme les confronte aux fonctions de Windows qu'elles remplacent :

— SID de services, calculés hors ligne, comparés à `LookupAccountNameW` pour les 273 services de la machine ;

— GUID, SID dans les deux sens, dates OLE, noms de propriétés (table de 1 698 entrées générée depuis le schéma de Windows) ;

— dates FAT, sur toutes les dates possibles ;

— heure d'été : décalage dans les deux sens sur les **141 fuseaux** de Windows, avec leurs règles année par année, à chaque transition de 1980 à 2035 (heure sautée, heure répétée, changement de règle au 1er janvier) ;

— lecture de la clé `TimeZoneInformation`, comparée à `GetTimeZoneInformation` ;

— règles de fuseau forgées au hasard : aucun débordement.

Écarts voulus, et documentés : WAC refuse un SID écrit que Windows tronquerait, et une date OLE non numérique pour laquelle Windows rend une heure dénuée de sens.

3.3 Harnais de robustesse

Un analyseur lit à des positions que le fichier déclare lui-même. Sur un fichier tronqué ou forgé, une lecture hors tampon ne plante pas d'elle-même : elle lit la mémoire voisine et la publie, dans un JSON valide. Les harnais de robustesse la font planter : chaque entrée est placée contre une page mémoire interdite, une fois à sa fin, une fois à son début. Un seul octet lu hors de l'entrée arrête le test. Un chien de garde arrête aussi un analyseur qui ne

termine pas.

Chaque fichier est analysé entier, dans ses troncatures, et en copies corrompues au hasard (graine fixe : un échec se reproduit à l'identique).

Programme	Fichiers d'une machine réelle	Entrées analysées
`lnk_test`	220 raccourcis	546 214
`parsers_test`	48 listes de sauts automatiques	175 585
	23 listes de sauts personnalisées	79 036
	307 Prefetch	1 246 491
	4 ruches du registre	16 207

Résultat : aucune lecture hors tampon, aucune boucle sans fin.

CHAPITRE 4

Le cycle automatique et ses contrôles

4.1 Le cycle

`vmtest/run-wac-test.sh` exécute, sans aucune intervention, une collecte complète dans une machine virtuelle Windows 11 (vrai NTFS, vrai registre) :

1. avec `--build`, compilation sans avertissement de WAC et des programmes de test ;
2. validation de l'extraction brute d'une ruche : `reg load` doit la refuser avant réparation et l'accepter après ;
3. collecte par WAC avec les droits SYSTEM (`--collect --events --binary`), puis conversion de cette collecte ; le code de retour est vérifié, et la trace est cherchée pour toute exception non traitée. Avant, le banc pose ses témoins : fichiers, valeurs de registre, événements, navigateurs, courrier, un script au chemin de plus de 260 caractères ;
4. rapatriement des sorties, du journal de collecte et de la liste des fichiers réellement présents dans la consigne ;
5. exécution de `check-json.py` ;
6. collecte seule (`--collect --events --binary`, ou `--binary-all`), qui ne doit rien convertir ; vérification de la consigne par `verify-exhibits.ps1` ; conversion d'une consigne dont une pièce a été retouchée, qui doit être refusée avant toute lecture ; conversion, contrôlée par `check-json.py` ; seconde conversion, dont les JSON doivent être identiques à la première, octet pour octet ;
7. contrôle d'innocuité : deux fichiers témoins que la collecte seule lit forcément gardent, avant et après elle, leurs trois dates et leur numéro USN (`harmlessness.txt`) ; une lecture de contrôle par Windows change la date d'accès de l'un d'eux, preuve que le contrôle sait échouer. Voir le *Dossier d'innocuité* ;
8. collecte pilotée par un `wac.yml` posé à côté de WAC, lancé sans option : elle doit suivre le fichier (artefacts coupés, aucun exécutable) et le consigner.

4.2 Les contrôles de `check-json.py`

Le script vérifie d'abord la validité JSON de chaque fichier et la forme des chemins Windows (un double échappement se voit après analyse). Il confronte ensuite des valeurs calculées par WAC à des valeurs indépendantes : une autre donnée de la même collecte, ou une source externe. Chaque contrôle ci-dessous a été écrit après un défaut réel, qu'il a pris.

Contrôle	Défaut qu'il a pris
Dates	
couples `X` / `XUtc`, listes comprises, désignant le même instant	double décalage horaire (sessions, raccourcis, date d'installation) ; heures d'exécution des Prefetch étiquetées `+02:00` au lieu de UTC (1 273 valeurs fausses de 2 h)
décalage de chaque date locale / base de fuseaux IANA	toutes les dates étiquetées avec le décalage du jour de collecte (106 dates d'hiver à `+02:00`) ; dates de changement d'heure de la ruche SYSTEM lues dans le mauvais format
aucune session antérieure au démarrage du système	un décalage de deux heures sur les sessions ; plus tard, une heure de démarrage en retard de 3,5 s
heure de démarrage / événement Kernel-General 12	deux sources indépendantes du même instant : 0,00 s d'écart
date ShimCache / date NTFS du fichier dans le manifeste	une date UTC traitée comme locale
aucun événement postérieur à la collecte	décalage ou mauvaise lecture d'une date d'événement
dates FAT sans fraction et à secondes paires ; dates Amcache lues en texte sans fraction	toutes les dates écrites avec sept chiffres de fraction, qui annonçaient une précision que la source n'a pas (808 dates)
dates de BAM, UserAssist, USBSTOR et Amcache / `taskkill` lancé par le harnais, Prefetch, `Get-PnpDeviceProperty`, en-têtes PE	ces quatre artefacts stockent de l'UTC, lu comme une heure locale : toutes leurs dates décalées de 2 h ; BAM et UserAssist vidés par une régression de la lecture des valeurs
Décodage	
noms des clés de sortie en anglais, sans abréviation	`NbVolumes`, `DateLocale`, `executionTime` : abréviation, mot français, minuscule initiale
identifiant et numéro de série d'une clé USB / `Get-PnpDevice`	aucune clé n'avait de numéro de série, de fabricant ni de produit : ils sont dans le nom des clés, pas dans des valeurs
empreinte d'un Prefetch / suffixe de son nom de fichier	65 empreintes fausses sur 270 (hexadécimal sans remplissage)
chemins `\VOLUME{guid}` des Prefetch résolus	20 066 chemins sur 20 066 non résolus
références `$MFT` des Prefetch / enregistrement de la `$MFT` de la même collecte (liens physiques compris)	séquence lue sur quatre octets, bits forts de l'entrée perdus : 14 940 références fausses sur 14 975

Contrôle	Défaut qu'il a pris
chemin et date de création de chaque pièce du volume système dans `mft.json` / manifeste de la consigne (descente des index de répertoires par l'extraction brute) ; numéros du journal des modifications croissants, aucun postérieur à la collecte	les fichiers à liens physiques (`System32` et `WinSxS`) n'apparaissaient que sous un nom (23 095 pièces sur 61 242) ; des répertoires nommés par leur alias 8.3 (`AM3B40~1.945`) quand leur nom long était dans un enregistrement d'extension (6 346 pièces) ; une pièce (`sru\SRU00B25.log`, 64 Kio) au contenu d'un autre fichier — les blocs `hbin` d'une ruche —, son enregistrement libéré par SRUM et réutilisé entre la lecture du répertoire et celle du fichier, lu sans vérifier sa séquence
témoins déposés par le cycle (historique PowerShell, fichier mis à la corbeille, entrée `hosts`, script de démarrage avec son SHA-256) / `powershell_history.json`, `recycle_bin.json`, `hosts.json`, `startup_folders.json` ; fichier marqué téléchargé, son flux relu par Windows / `zone_identifiers.json`, lu dans la `$MFT` ; sections de `setupapi.dev.log` comptées par Windows / `setupapi.json` ; noms des secrets LSA, `PolAdtEv` et emplacements `NL$` lus en direct par Windows ; droits des comptes (`AccountRights`) face à `secedit /export` (33 droits) / `security.json` ; valeur Run `REG_EXPAND_SZ` de la ruche DEFAULT / `run.json` ; témoins de persistance (IFEO, SilentProcessExit, Active Setup, détournement COM de l'utilisateur) et valeurs de Windows lues en direct (Userinit, paquets LSA, moniteurs d'impression) / `persistence.json` ; témoins du lot 2 (document approuvé, chemin tapé, recherche, proxy, règle de pare-feu) et racines de la machine listées par Windows, SHA-1 de chacune = son empreinte / JSON du lot 2	contrôle ajouté avec ces artefacts

Contrôle	Défaut qu'il a pris
programme témoin dans la clé `Uninstall` de la machine, et nombre de programmes de chaque clé (machine, 32 bits, utilisateur) listés par Windows / `installed_programs.json`	a montré que les ruches des utilisateurs n'étaient copiées qu'avec la clé `registry` : sans elle, chemins tapés et recherches sortaient vides
exclusions de Defender témoins (chemin, extension, processus) et liste de `Get-MpPreference` / `defender_configuration.json`	contrôle ajouté avec l'artefact
fichier témoin ajouté aux documents récents par le shell lui-même (`SHAddToRecentDocs`), en tête de sa liste / `explorer_recent_docs.json`	contrôle ajouté avec l'artefact
partage témoin mappé par l'utilisateur, et entrées de `MountPoints2` listées par le registre en ligne / `mount_points.json`	a montré que les entrées de `CPC\Volume` sont volatiles, absentes du fichier de ruche
profils et passerelles listés par le registre en ligne ; dernière connexion dans le journal `NetworkProfile` (événement 10000, UTC) / `known_networks.json`	a établi que ces dates sont en heure locale
réglages lus dans le registre en ligne ; dernier arrêt dans le journal System (Kernel-General 13) / `OperatingSystem.json` (`Settings`, `ShutdownTime`)	contrôle ajouté avec ces champs
valeurs de `StartupApproved` et `RunNotification` lues dans le registre en ligne / `run.json`	contrôle ajouté avec ces clés
mode de compatibilité témoin ; noms de chaque source lus dans le registre en ligne ; nombre d'éléments épinglés ; programmes de chaque `OpenWithList` dans l'ordre de `MRUList`, extension témoin comprise / `program_usage.json`	a établi que `FavoritesResolve` est une suite de raccourcis complets
périphériques USB et SCSI et leurs trois dates selon `Get-PnpDeviceProperty` / `devices.json`	contrôle ajouté avec l'artefact

Contrôle	Défaut qu'il a pris
partage témoin ; partages de `Get-SmbShare` et droits de `Get-SmbShareAccess` / `network_shares.json` (descripteur de sécurité décodé)	contrôle ajouté avec l'artefact
cartes de `Get-NetAdapter` : description, adresse matérielle, adresse DHCP / `network_configuration.json`	contrôle ajouté avec l'artefact
journaux de `Get-WinEvent -ListLog` et leur état actif / `event_log_configuration.json` (490 journaux) ; sessions de `Get-AutologgerConfig` et leur `Start` (39 sessions)	contrôle ajouté avec l'artefact
`ActivitiesCache.db` de Windows 10 (données de test de plaso, 112 activités) lu par le `sqlite3` de Python, chaque colonne convertie indépendamment ; base réelle de la VM, journal `-wal` rejoué / `activities_cache.json`	contrôle ajouté avec l'artefact ; une date décalée d'une heure est prise (113 échecs)
document témoin indexé par Windows ; requête de Windows sur son propre index (OLE DB, `Search.CollatorDSO`) : URL, modification, taille de chaque élément / `search_index.json` (515 éléments identiques)	contrôle ajouté avec l'artefact
PST public épinglé (EDRM Enron) déposé dans les `Outlook Files` de « wac », lu par pypff (libpff) ; OST d'Outlook 2013 et PST chiffrés du corpus java-libpst (`pst_test`) / `emails.json` (les 246 messages que lit pypff identiques : objet, expéditeur, remise, 65 pièces jointes par nom, taille et SHA-256, corps ; 250 lus par WAC)	contrôle ajouté avec l'artefact

Contrôle	Défaut qu'il a pris
message relevé par Thunderbird lui-même (serveur POP3 local, objet en mot encodé, pièce jointe base64 au nom RFC 2231), boîte mbox lue par les modules `mailbox` et `email` de Python / `emails.json` (objet et pièce jointe par nom et SHA-256 identiques)	contrôle ajouté avec l'artefact ; Thunderbird ne relève pas sans authentification, et en NTLM demande aussi un mot de passe
image vue par « wac » dans l'Explorateur, son `System.Thumbnail CacheId` donné par Windows ; les deux sommes de contrôle CRC-64 que Windows écrit pour chaque entrée, recalculées / `thumbcache.json` (835 entrées de 9 caches, toutes les sommes d'en-tête et les 777 sommes de données identiques ; la miniature du témoin retrouvée, exportée avec son SHA-256)	contrôle ajouté avec l'artefact
session RDP réelle de « wac » vers 127.0.0.2 avec un compte de test, fermée comme un utilisateur le ferait ; serveur témoin de valeurs connues ; tuiles du cache lues par bmc-tools (ANSSI) ; événements entrants listés par `Get-WinEvent` / `rdp_client.json`, `rdp_server.json` (4 tuiles aux mêmes pixels, 3 traces entrantes, même utilisateur et même source)	contrôle ajouté avec l'artefact ; le cache n'est écrit qu'à la fermeture du client, et Windows demande désormais une confirmation à l'ouverture d'un fichier `.rdp`
message rendu par Windows lui-même (`EvtFormatMessage`, par `Get-WinEvent`) pour chaque sorte d'événement de la VM — fournisseur, identifiant, version, qualificatifs — sur le même enregistrement / `EvtEventMessage` de `events.json` (887 sortes sur 898 identiques)	messages de 28 % des événements seulement : pas des descripteurs du `WEVT_TEMPLATE` déduit de la taille du bloc (52 octets au lieu de 48) ; sources classiques ignorées ; tables UTF-8 lues octet par octet ; satellite `en-US` préféré au `fr-FR` faute de lire la langue dans SYSTEM ; espaces des données rognés ; valeurs non typées (tables, drapeaux, codes d'erreur) ; données décalées par un tableau ou une feuille vide ; manifestes déplacés dans `SystemResources` ; en collecte séparée, sources classiques et tables du système non prélevées, `CurrentControlSet` jamais ouvert

Contrôle	Défaut qu'il a pris
audit des accès de Windows (règle d'audit de lecture sur la consigne de la collecte séparée, événements 4663 du journal Security) / lectures de la consigne par `WAC.exe` pendant la conversion, comparées à l'instant de sa recopie (`conversion.json`) : 1 458 lectures, toutes avant, aucune après	une quinzaine de modules lisaient les pièces dans la consigne, la `$MFT`, le journal des modifications et les binaires y restaient « lus sur place »
raccourci témoin du Bureau public et raccourci épinglé à la barre des tâches (cible, arguments, emplacement) ; raccourcis listés par Windows dans les menus Démarrer, Bureaux et Quick Launch / `shortcuts.json` (93 raccourcis avec ceux du profil Default et du Bureau redirigé, ni plus ni moins)	contrôle ajouté avec l'artefact
valeur Run posée dans le `NTUSER.DAT` du profil Default / `run.json`, SID `S-0-0-0` et `Profile Default`	contrôle ajouté avec le profil Default
condensat NTFS de chaque descripteur de `$Secure:$SDS`, recalculé (2 818 descripteurs) ; propriétaire de 41 fichiers selon `Get-Acl`, retrouvé par le `SecurityId` de `mft.json` ; ACE Everyone posée par `icacls` / `security_descriptors.json`	contrôle ajouté avec l'artefact
numéro de série, tailles de secteur, de cluster et d'enregistrement, clusters de la `$MFT` et de son miroir, identifiant, taille, pas et premier USN du journal, donnés par Windows (`DeviceIoControl`) / `ntfs_volumes.json`, lus en brut dans `$Boot` et `$Max` (2 volumes)	contrôle ajouté avec l'artefact
traces WPP de Defender listées par Windows / manifeste de la consigne, copiées brutes	contrôle ajouté avec la copie

Contrôle	Défaut qu'il a pris
fichiers de clés maîtresses DPAPI listés par Windows, GUID de l'en-tête égal au nom ; clé désignée par chaque `Preferred` et son expiration, lus par Windows / `dpapi_master_keys.json` (5 clés, 3 dossiers)	contrôle ajouté avec l'artefact
valeurs de `UserShellFolders` de « wac » lues non développées par Windows, son Bureau redirigé vers `OneDrive\Desktop` / `user_shell_folders.json` (20 valeurs) ; raccourci posé sur le Bureau redirigé / `shortcuts.json`	contrôle ajouté avec l'artefact
URL, visites (URL, instant, transition) et téléchargements de la base `History` d'Edge, relue par le SQLite de Python / `web_client.json` (5 URL, 7 visites)	contrôle ajouté avec l'artefact
deux `TypedURLs` de « wac » et leurs instants dans `TypedURLsTime` / `web_client.json`	contrôle ajouté avec l'artefact
entrées de `WebCacheV01.dat` relues par `dissect.esedb` (URL, dates UTC ; heure locale des conteneurs `MSHist` convertie par `zoneinfo`) / `web_client.json` (28 entrées)	le décalage de 2 h de l'heure locale de `MSHist`, relevé avant d'écrire le décodeur
lignes de `Login Data` et de `Network\Cookies` d'Edge relues par le SQLite de Python, mêmes sites / `web_client.json` (0 identifiant, 10 cookies)	contrôle ajouté avec l'artefact
navigation réelle de « wac » (Chrome et Firefox installés par `install-browsers.sh`) : visite d'une page témoin, recherche Google, téléchargement servi en pièce jointe par Chrome et par Firefox, HTA lancée par `mshta.exe` ; bases de Chrome et `places.sqlite` de Firefox (journal WAL rejoué) relues par le SQLite de Python / `web_client.json`	contrôle ajouté avec les navigateurs de la VM

Contrôle	Défaut qu'il a pris
fichiers de session de Chrome et d'Edge (`Session_*`, `Tabs_*`) confrontés à leur `History` relu par le SQLite de Python ; sessions de Firefox décompressées par le module `lz4` de Python ; `Favicons` et `favicons.sqlite` relus par le SQLite de Python / `web_client.json` (108 pages de session Chromium : les 57 que l'historique connaît portent l'instant d'une visite, 55 à la microseconde, 2 une navigation interne 2,5 s plus tard ; 54 pages Firefox ; 28 couples page et icône)	contrôle ajouté avec l'artefact ; le format, vérifié sur les fichiers réels avant d'écrire le test
HTA servie par le serveur témoin et lancée par `mshta.exe` : son entrée du WebCache nomme sa copie en cache, collectée, même SHA-256 que le contenu servi / `web_client.json`, manifeste	contrôle ajouté avec la collecte du cache
enregistrements de quatre `index.dat` réels (données de test de plaso) lus par `pymsiecf` / décodeur des `index.dat` (1 110 enregistrements identiques)	un enregistrement récupéré en partie écrasé rendait les données d'un autre
téléchargement par le vrai Internet Explorer 11 (objet COM, « Enregistrer », fermeture normale) / `web_client.json` (`iedownload` : URL, chemin enregistré, taille, adresse du serveur)	la taille et l'adresse lues à de mauvais décalages, que le test unitaire, construit sur les mêmes, ne voyait pas
entrées DestList / raccourcis des listes de sauts	les 15 premières entrées de chaque liste perdues (573 raccourcis publiés sur 1 522)
nom de machine des événements / `OperatingSystem.json`	contrôle du décodage BinXML, sur deux sources sans lien
identifiants d'enregistrement uniques par journal	relecture d'un bloc périmé
références `%%nnnn` non résolues dans les messages	fichier de paramètres d'un fournisseur non chargé (3 780 messages)
RID retrouvé à la fin du SID reconstruit	lecture de la base SAM

Contrôle	Défaut qu'il a pris
condensat NT d'un compte témoin à mot de passe connu, et l'ancien dans l'historique / `users.json`	MD4 du mot de passe en UTF-16LE, recalculé à part (chaîne clé de démarrage / AES / DES vérifiée de bout en bout)
membres directs de chaque groupe local, compte témoin dans les Administrateurs / `local_groups.json`	`Get-LocalGroupMember` sur les 21 groupes
questions de sécurité (`Security Questions`) d'un compte témoin / `users.json`	`ResetData` écrit puis relu, questions et réponses
états et types des services, présence des pilotes	filtres d'énumération confondus avec des états
aucun processus ne porte `WAC.exe` parmi ses modules	modules de l'outil attribués au processus Idle
références `$MFT` plausibles	décodage des blocs d'extension des raccourcis
montages / lecture de la clé par Windows (`reg query`) et partitions (`Get-Partition`)	6 montages sur 7 publiés comme « \ » (chemins de périphérique décodés en ANSI)
noms des SID / `Get-LocalUser`, `Get-LocalGroup`	noms demandés au système en marche, voire au contrôleur de domaine, et traduits dans la langue du poste (« Système »)
GUID inconnus de la table / trois GUID enregistrés par le harnais, un par source	« Unmapped GUID » publié comme un nom, dont 18 gestionnaires COM de tâches de la VM, nommés dans sa ruche
Consigne et rejeu	
sceau / empreinte réelle du manifeste	retouche du manifeste
trois empreintes par pièce, comptes cohérents	pièce non identifiée
fichiers présents dans la consigne / manifeste	209 pièces ajoutées après le scellement, que tous les autres contrôles laissaient passer
aucune ruche à la fois rejouée et réparée	rejeu qui n'aligne pas les séquences
un journal d'annulation pour chaque rejeu	copie brute impossible à reconstituer
Lecture brute, collecte et conversion séparées	
SHA-256 de fichiers lus en brut / `Get-FileHash`	garde-fou de la lecture par lots de clusters contigus
dates des Prefetch / dates données par Windows	0 sur 335 conformes : dates de la copie de travail, soit la minute de la collecte

Contrôle	Défaut qu'il a pris
exécutables de System32 et SysWOW64 / manifeste d'un `--collect --binary`	6 446 manquants sur 8 540 (`$INDEX_ROOT` en enregistrement d'extension, noms de liens physiques non déclarés)
catalogues cités dans un verdict / pièces de la consigne	170 sur 170 absents (libellé « catalog » cherché comme « catalogue »)
verdicts Microsoft de WAC / `Get-AuthenticodeSignature`	garde-fou : un mauvais verdict laisserait un binaire hors de la consigne
copie d'un paquet du Store : fichier intact, modifié, ajouté	vérification par la carte des blocs signée
dates d'un script reculées à 2001 / `$STANDARD_INFORMATION` et `$FILE_NAME`	garde-fou : une falsification doit rester visible
clés de récupération / en-têtes PE lus par PowerShell	garde-fou du contenu récupérable
`--binary-all` : copie et verdict de chaque exécutable	contrôle des paquets trop strict en `--binary-all`
pièce retouchée / refus de la conversion	garde-fou de la vérification avant lecture
deux conversions d'une même collecte	dates lues sur la copie de travail, refaite à chaque conversion

4.3 Résultat de référence

Dernier cycle, le 9 octobre 2026, sur la version 2.0.0 (Windows 11, fuseau de Paris) : 472 contrôles passés, aucun échec.

— **collecte puis conversion** (`--collect --events --binary`, collecte 7 min, conversion 1 min 47 s) : 72 fichiers JSON valides, USBSTOR compris (le harnais branche deux clés USB virtuelles à chaque cycle, si bien qu'une machine de test recréée les connaît aussi) ; 226 contrôles passés, dont 59 107 dates locales confrontées à la base IANA et 211 972 événements décodés dans 405 journaux, dont 98,5 % des sortes d'événements avec le texte que rend Windows ; 6 014 fichiers dans la consigne, exactement ceux du manifeste, sceau conforme ; un script au chemin de 261 caractères consigné avec l'empreinte que donne Windows ;

— **collecte seule** (`--collect --events --binary`, 7 min 30 s) : rien converti ; 49 226 exécutables lus, 28 180 authentifiés comme Microsoft et 328 tiers validés par leur chaîne, non copiés ; 5 430 contenus déjà consignés ; 2 597 prélevés (771 Mo) ; 11 515 pièces vérifiées par `verify-exhibits.ps1` ; une pièce retouchée refusée avant toute lecture ; contrôle d'innocuité conforme ;

— **conversion** (`--convert`, 1 min 43 s) : 243 contrôles passés, dont les 8 537 exécutables de System32 et SysWOW64 présents au manifeste, 625 clés de récupération égales

aux en-têtes PE, un script aux dates falsifiées démasqué (`$STANDARD_INFORMATION` de 2001, `$FILE_NAME` du jour), la vérification des paquets du Store ; une seconde conversion donne des JSON identiques, octet pour octet ;

— **collecte pilotée par `wac.yml`** : le fichier est suivi et consigné.

CHAPITRE 5

Les recoupements sur une collecte réelle

Une machine virtuelle neuve ne produit ni fragmentation, ni listes de sauts anciennes, ni noms de fichiers exotiques, ni journaux abîmés. Plusieurs défauts ne se sont révélés que sur une vraie machine, confrontée à une source indépendante :

Recoupement	Ce qu'il a montré
listes de sauts / `olefile`	les flux s'appellent « 1 » à « f », et non « 01 » à « 0f »
cible d'un raccourci / entrée DestList	une page de code fausse et un chemin Unicode ignoré
journaux / `python-evtx`	un bloc de signature fausse arrêtait la lecture de tout le fichier (14 enregistrements lus sur 270)
blocs EVTX / leur CRC32	contrôle ponctuel : les 1 505 blocs des 404 journaux de la machine de test sont intègres
ruches / `offreg.dll`	1 273 652 clés identiques
binaires / `Get-AuthenticodeSignature`	aucune authentification abusive
ancienne sortie / nouvelle	décodage des mêmes fichiers par le code courant, comparé après normalisation

> **À retenir**
> Une collecte réelle contient des données personnelles. Elle se traite en local ; une copie temporaire dans la machine virtuelle ou un dossier de travail est effacée après usage, et rien n'en entre dans le dépôt des sources.

CHAPITRE 6

Ce que ces contrôles garantissent

6.1 Garanties

— **Intégrité des pièces** : chaque pièce est identifiée par trois empreintes, la copie de travail est vérifiée contre la lecture du volume, le manifeste est scellé, et la consigne n'est jamais modifiée (`consigne_test`, contrôles du cycle).

— **Aucune écriture sur le système examiné**, hors les traces inévitables d'une collecte sur un système en marche, toutes consignées dans `investigation.json` avec la trace attendue.

— **Exactitude des décodeurs** : pour ceux qui ont une référence (lecteur de ruches, décompressions, empreintes, conversions, fuseaux horaires, authenticité des binaires), résultats identiques à la référence sur les volumes indiqués.

— **Robustesse** : aucun analyseur de raccourcis, listes de sauts, Prefetch ou ruches ne lit hors de son tampon ni ne boucle, sur plus de deux millions d'entrées tronquées ou corrompues.

— **Cohérence des sorties** : chaque recoupement possible entre deux artefacts, ou entre un artefact et une source externe, est vérifié à chaque cycle.

— **Indépendance vis-à-vis de la machine examinée** : WAC n'emprunte à celle-ci aucune bibliothèque pour interpréter les preuves.

6.2 Limites

> **Limite**
> Ces contrôles ne prouvent pas l'absence de défaut. Ils prouvent que les défauts qu'ils visent sont absents, sur les données qu'ils ont vues.

— **Complétude** : un artefact vide ne se distingue pas d'une machine sans trace. Seule la comparaison entre deux collectes, ou avec une machine dont on connaît le contenu, la mesure.

— **Données que la machine de test ne contient pas** : appareils MTP, fichiers éclatés sur plusieurs enregistrements de la `$MFT` (`$ATTRIBUTE_LIST`), très gros volumes. Ils sont à valider sur des systèmes réels.

— **Artefacts sans référence** : quand aucun outil indépendant ne lit un format, sa justesse

repose sur les recoupements internes et sur la documentation publique du format.

— **Fuseaux horaires** : la confrontation à la base IANA couvre les fuseaux dont les règles concordent avec celles de Windows sur la période vérifiée (Europe de l'Ouest, États-Unis, UTC) ; les autres sont vérifiés contre Windows seulement.

— **Compilation sous Windows** : `build-windows.cmd` lance `build-windows.sh` dans le bash de Git for Windows, avec le gcc 13.2.0 de MinGW-Builds (modèle de fils « win32 », msvcrt). Le cycle complet a été passé une fois avec l'exécutable compilé ainsi, sans écart avec celui compilé sous Linux ; les cycles courants portent sur la compilation Linux, celle qui est livrée. Un gcc au modèle « posix » (MSYS2) est refusé : il ajoute 38 importations à `WAC.exe`.